

SIH26102 — Research Brief

AI-Powered Anomaly, Fraud & Inefficiency Detection for MPLAD Scheme Implementation

Prepared for: Team Neural Nova — DYPIU Internal SIH 2026 Round **Problem**

Statement: SIH26102 | **Organization:** Ministry of Statistics and Programme Implementation (MoSPI) **Team:** Sreeram Sushanth (lead — architecture, AI/ML, backend, integration, pitching), Aditya (execution partner), Shagun (frontend), Sarthak & Manav (modules), Gayatri (testing, with Manav)

1. Executive Summary

MPLADS (Members of Parliament Local Area Development Scheme) channels ₹5 crore per MP per year — roughly ₹2,700+ crore annually across 543 Lok Sabha constituencies plus Rajya Sabha and nominated members — into local infrastructure works recommended by MPs and executed by District Authorities and Implementing Agencies. Since April 2023, the entire fund-flow and record-keeping process runs through a government digital platform called **eSAKSHI** (built with State Bank of India), replacing the earlier physical, paper-and-bank-account system.

eSAKSHI solved the *record-keeping and fund-flow* problem. It did **not** solve the *anomaly detection and audit-prioritization* problem — it digitized the paperwork, but a human still has to notice that a water tank costs 3x the district median, or that the same "completed" photo appears on two different works. SIH26102 asks teams to build the layer that sits on top of eSAKSHI-style data and tells auditors **where to look first**.

This is a well-trodden PS this cycle — dozens of public teams have already built prototypes, and several converge on a near-identical architecture (see Section 4). That convergence is actually useful intel: it tells you what a judge has *already seen five times today*, and where the real differentiation room is.

2. Background: The Scheme & Its Digital Backbone

MPLADS mechanics:

- MP recommends a work → District Authority does feasibility checks and sanctions it → an Implementing Agency (a PWD, municipal body, NGO, etc.) executes it and raises payment requests → funds are released through eSAKSHI, integrated with the Central Sector Scheme fund-flow mechanism.
- Each MP gets an annual entitlement (₹5 crore/year), and works are meant to complete within roughly one year of sanction.

eSAKSHI portal — what it already does:

- Onboarded all States/UTs; single login for MPs, MoSPI, State/District Authorities, and Implementing Agencies.
- Tracks the full lifecycle: recommendation → sanction → vendor identification → physical inspection → payment.
- Since March 2026, a **revamped public dashboard** lets citizens drill into works recommended/sanctioned/completed per constituency, and mandates geo-tagged photo uploads at construction stages — explicitly framed by MoSPI as enabling "Citizen Audit."
- MoSPI runs monthly review meetings with States/UTs and shares pendency data: works unsanctioned beyond 45 days, works incomplete after 1 year, works with no payment 3 months post-sanction.

What eSAKSHI does *not* do (this is your opening):

- It reports pendency and delay *after the fact*, in review meetings — it doesn't compute a live risk score or auto-flag anomalies at the moment a fund request is raised.
- Publicly released data is largely at the constituency/MP-allocation level, not deep contractor-level sub-ledgers — so contractor-nexus detection (see Section 6) currently has to be built on inference and synthetic augmentation, not a clean contractor API.
- No cross-referencing against peer costs (unit cost per sq ft/km by terrain type), no duplicate-photo detection, no contractor concentration analysis, no geospatial proximity conflict detection.
- Known operational gaps outside the software itself: digital literacy gaps among some stakeholders, connectivity issues for field inspectors in remote areas, and general resistance to workflow change.

Sources: PIB press release on eSAKSHI; MoSPI eSAKSHI dashboard; Policy Edge coverage of the March 2026 public-dashboard revamp; IASPOINT explainer.

3. Target Audience / Stakeholders

This PS has **five distinct user personas**, and a strong submission names all five explicitly (most weak submissions only build for one):

Stakeholder	What they need from the system
District Collectors / Audit Teams	A prioritized worklist — "inspect these 12 works first" — not a raw data dump. This is the primary user.
MoSPI (Ministry) officials	Aggregate, cross-state dashboards; trend detection; policy-level reporting for the monthly review meetings they already run.
MPs / their office staff	Visibility into their own constituency's works and risk flags, without feeling accused.
Implementing Agencies / Contractors	Indirectly affected — the system's outputs shape who gets scrutinized. Needs to be "explainable," not accusatory (see Section 7).
Citizens	The March 2026 eSAKSHI revamp explicitly invites "Citizen Audit." A grievance/complaint channel that feeds back into the risk engine is now expected, not optional.

4. Competitive Landscape — What Other Teams Are Building

Since this PS is public and heavily attempted, I pulled several public prototypes (GitHub repos built for this exact PS) to map the convergence zone. **Assume judges have seen most of this already.**

The now-standard architecture pattern

Almost every serious submission converges on the same shape:

eSAKSHI/public data → ingestion & validation → relational DB (SQLite/Postgres+PostGIS)

→ N parallel "detection engines" → fusion into single 0–100 risk score

→ explainability layer (SHAP-style "why flagged") → role-based dashboards + PDF audit dossier

The near-universal detection engine set

Engine	What it catches	Typical method
Financial outlier	Costs way above peer/district/terrain norm	Z-score, IQR, Isolation Forest
Contractor nexus	One contractor/entity monopolizing works across districts	Graph analytics (NetworkX), bipartite centrality
Spatial/GIS	Two "different" works suspiciously close together (<150m)	Haversine geodesic distance
Split-tender	Work artificially chunked to duck a procurement approval threshold	Threshold-window aggregation
Evidence/photo	Same "completion" photo reused across different works	Perceptual hashing (dHash/pHash)
NLP text similarity	Copy-pasted or near-identical work descriptions	TF-IDF + cosine similarity

Engine	What it catches	Typical method
Temporal/velocity	Payment released before physical progress, or work stalled	Milestone-latency and disbursement-velocity checks

Common differentiators teams are adding on top

- **Sanction-time interception** — flagging an over-priced fund request *at the moment it's submitted*, before money moves, rather than auditing after the fact. This is the single most "innovative" add-on appearing across the strongest repos, and matches exactly what your RDBMS mentor called out as innovative — worth leaning into hard.
- Environmental compliance checks (tree-cutting/demolition limits).
- 3D/visual "future completion" renders for citizen-facing transparency.
- Satellite/drone imagery cross-checks (ISRO Bhuvan) — flagged by multiple teams as *future scope*, not built, because of integration cost. This is a credible "roadmap" slide item, not a hackathon-weekend build.
- A defensible-language layer: never say "fraud detected," always say "anomaly-priority score, verification required" — several teams explicitly document this as a presentation strategy, since MoSPI/judges will not want a tool that makes unaccountable accusations.

Where the field is weak (your opening)

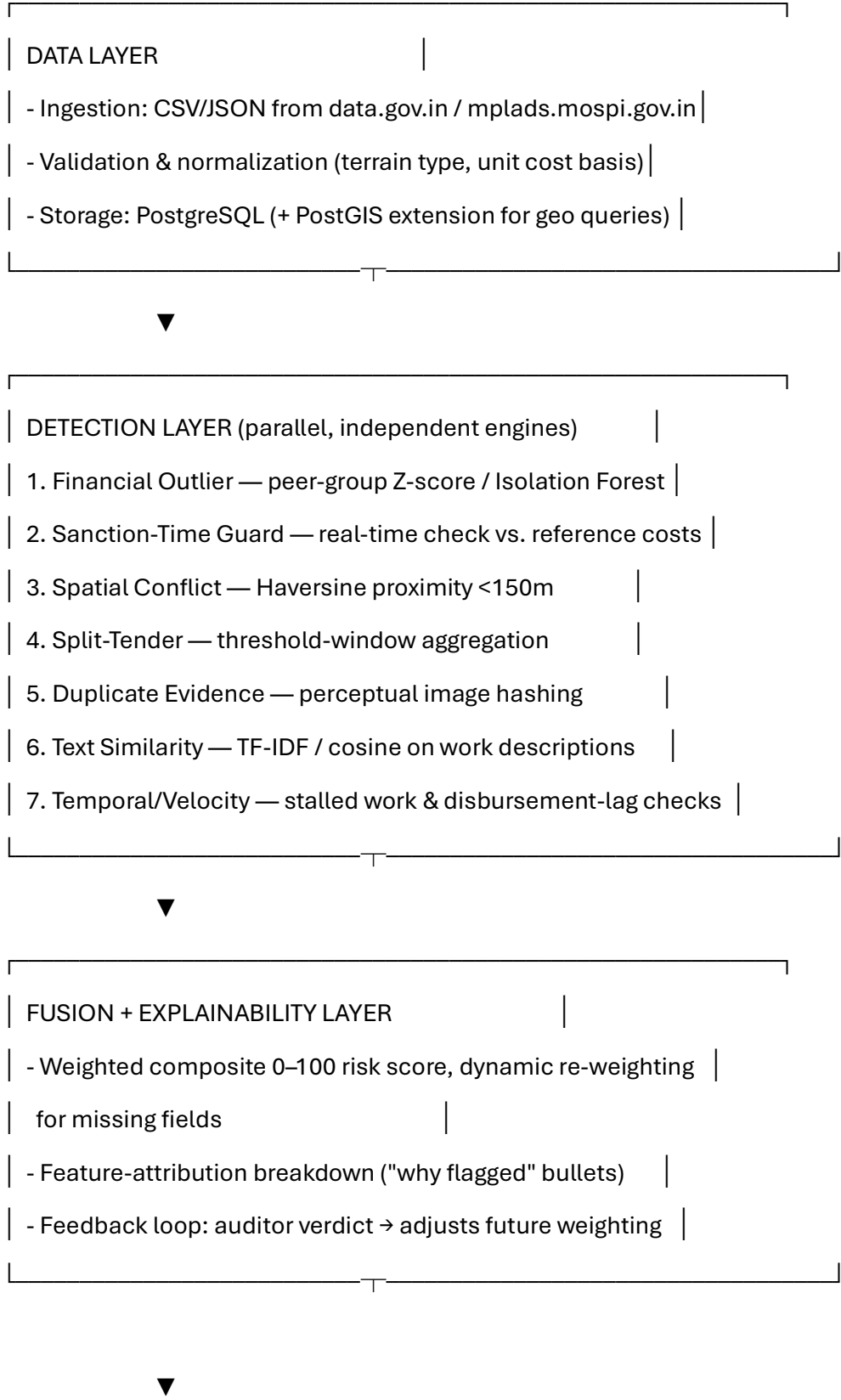
- Almost everyone uses **synthetic or lightly-augmented demo data** because granular contractor-level data isn't publicly exposed — few have actually scraped/normalized the real public eSAKSHI dataset end-to-end. One team (sujoy18207/SIH2026) explicitly built on ~60,000+ real MPLADS records scraped from the official portal — that's a real differentiator if you can pull it off, and a credibility bar if you can't match it.
- Citizen grievance loops are usually bolted on as an afterthought (a simple form), not actually fed back into re-scoring or risk-weighting.
- Very few teams show a **feedback/learning loop** — i.e., what happens after a human auditor marks something "verified — not fraud" or "confirmed anomaly." Systems mostly stop at "flag and rank," with no mechanism to improve the model from audit outcomes over time.
- Offline/low-connectivity field use (inspectors in rural areas with no signal) is acknowledged as a gap by teams but rarely solved.

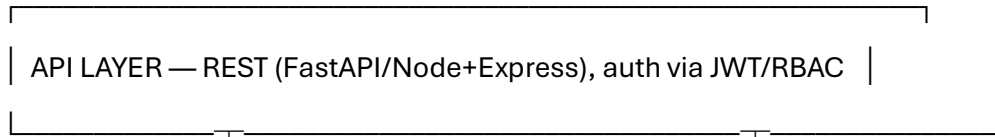
5. Gaps & Improvement Opportunities for Your Build

Given the field above, here's where Neural Nova can differentiate without over-scoping for a hackathon timeline:

1. **Sanction-time interception, not just post-hoc audit.** This is explicitly the newest and most-praised feature across recent repos, and it's a genuinely different mode from "detect after the fact" — flip the system from forensic to preventive. Your mentor calling this "innovative" tracks with what's landing well elsewhere.
 2. **A real (if partial) feedback loop.** When a collector marks a flagged work "verified clean" or "confirmed issue," feed that back to adjust engine weights or at least maintain a precision/recall dashboard over time. Almost nobody is showing this — it signals you understand this is a *decision-support system*, not a one-shot classifier.
 3. **Citizen grievances as a first-class signal, not a form.** Route citizen complaints into the same risk-scoring pipeline (geo-matched to the relevant work) rather than a separate unconnected table.
 4. **Explainability as UX, not a stat.** The strongest repos have a visible "WHY FLAGGED?" breakdown. Make this the centerpiece of your demo — judges respond well to a human-readable explanation over a black-box percentage.
 5. **Be honest about real vs. synthetic data,** and design your data layer so real eSAKSHI/data.gov.in records can be swapped in without rearchitecting — this is a credibility point judges will probe.
-

6. Proposed Internal Architecture





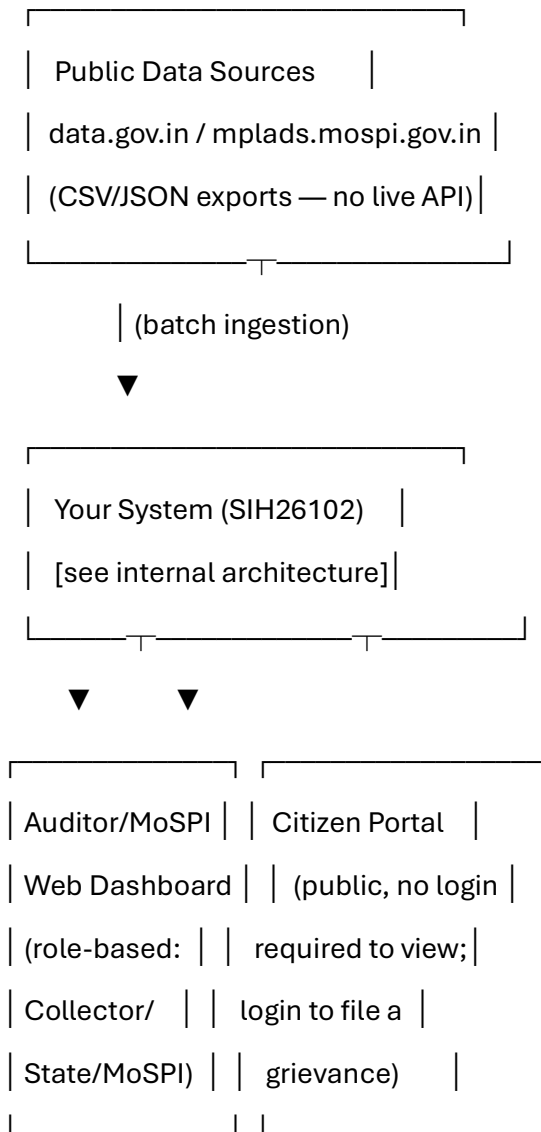
Auditor/Collector Dashboard Citizen Portal

(prioritized worklist, GIS map, contractor graph, PDF dossier) (view local works, submit geo-tagged grievances)

Recommended stack (given your team's known skills — React/Tailwind, Python, Supabase/Firebase experience):

- Frontend: React + Tailwind (Shagun) — matches what's already in your toolkit.
 - Backend: Python (FastAPI) for the ML/detection engines — Python is your strongest shared language and has the best ecosystem for scikit-learn, NetworkX, image hashing.
 - DB: Postgres with PostGIS if you want real geospatial queries; SQLite is fine for a demo-scale dataset if PostGIS setup risks eating hackathon time.
 - Keep it within free tiers per your stated hackathon cost preference — Supabase (Postgres+PostGIS is supported on Supabase) is a strong fit here since you already know it.
-

7. Proposed External Architecture (System Boundary & Integrations)



Key honesty point for your pitch: there is currently **no public live API** for eSAKSHI/MPLADS — only periodic data.gov.in/portal CSV exports. Don't pitch "real-time integration with eSAKSHI" as if it's a given; pitch your system as an **overlay that ingests the available public exports today and is architected to plug into a live eSAKSHI API if/when MoSPI exposes one** — this is both accurate and shows systems-thinking to judges (who are MoSPI-adjacent and will know the data limitation).

8. Pros / Cons of Key Architectural Choices

Choice	Pros	Cons
Full 7-engine detection suite (matches the field)	Comprehensive, judge-familiar, shows breadth	Hard to build <i>well</i> in hackathon time; risk of 7 shallow engines instead of 3 deep ones
Sanction-time interception (differentiator)	Genuinely novel angle, matches mentor's "innovative" feedback, prevents rather than audits	Needs a believable reference-cost dataset; harder to demo convincingly without real cost benchmarks
Real scraped eSAKSHI data vs. synthetic	Far higher credibility; judges can fact-check it	Scraping/cleaning eats significant time; portal structure may block easy scraping
PostGIS for real geospatial queries	Correct, scalable, "production-grade"	Extra setup complexity/time vs. plain lat/long distance math in Python
Feedback loop (auditor verdict → re-weighting)	Shows you understand this as a continuous system, not a one-shot tool — rare among competitors	Needs a believable UI flow + enough synthetic "verdict" history to demo meaningfully
Full RBAC across 4+ roles	Realistic, matches gov requirements	More auth/access-control code than a hackathon weekend easily affords

Recommendation given your team size (6) and roster: Don't try to build all 7 engines at production depth. Build **3 engines deeply** (Financial Outlier + Sanction-Time Guard + Spatial Conflict — these are the ones judges will most directly test/question) with real explainability, and **stub the remaining 2–3 with clearly-labeled "Phase 2" placeholders** in the UI rather than faking depth you don't have. This lines up with your

own stated hackathon principle: back every visible claim with real functionality or mark it prototype/simulated.

9. Suggested Task Split (mapped to your roster)

- **You (lead):** Architecture, fusion/scoring engine, sanction-time guard (the differentiator), integration, pitch narrative.
 - **Aditya:** Core detection engines (financial outlier, spatial conflict) alongside you.
 - **Shagun:** Frontend — auditor dashboard, GIS map view, citizen portal.
 - **Sarthak & Manav:** Well-scoped, bounded modules — e.g., data ingestion/cleaning pipeline for Sarthak, duplicate-photo hashing module for Manav (self-contained, testable in isolation — matches your note that they need simpler, well-defined pieces).
 - **Gayatri (with Manav):** Testing — edge cases, synthetic dataset validation, breaking the detection thresholds, verifying the "why flagged" explanations actually match the underlying data.
-

10. Risks & Open Questions to Resolve Before Building

1. **Data sourcing** — confirm whether you'll scrape real data.gov.in/mplads.mospi.gov.in exports or use synthetic data from day one. This decision shapes almost everything downstream and should be locked before the Sept 24 PPT deadline.
 2. **Scope discipline** — the field shows a strong pull toward "add one more engine." Resist it; depth on 3 engines beats shallow coverage of 7 in a live demo/Q&A.
 3. **Framing language** — bake "anomaly-priority score, not fraud accusation" into your PPT and demo script from the start; this is now a well-established norm in this PS's submissions and judges likely expect it.
 4. **Mentor check-in** — since your RDBMS mentor already flagged the idea as innovative, get her specific read on the sanction-time-interception angle and the feedback-loop idea before you lock the PPT — she may have visibility into what MoSPI evaluators specifically care about.
-

Sources

- PIB India — eSAKSHI portal press release
- MoSPI eSAKSHI public dashboard (mplads.mospi.gov.in)
- Policy Edge — "Revamped Public Dashboard of MPLADS eSAKSHI Portal" (March 2026)
- IASPOINT — MPLADS eSAKSHI Portal explainer
- GS Times IAS-PCS — MPLADS–eSAKSHI web portal overview
- Public GitHub repositories built for SIH26102 (multiple teams, 2026 cycle) — architecture and feature patterns synthesized across submissions
- SIH 2026 official problem statement catalogue (theme: Miscellaneous/Smart Automation, Organization: MoSPI)